INDIVIDUAL PROJECT REPORT

Review 2 — Design, Development & Progress (14/9/2026)

AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry

Student	Register No.	Team No.	My role
Sreehari	24BDS0417	_____	Sensing layer, hardware and recovery of lost data

Team: Laksh Ahuja (24BDS0212), Anvay Sharma (24BCE0824), Sreehari (24BDS0417). **Repository:** https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK

Summary

Since Review 1 I built the sensor-node simulator of our prototype and the function that fills gaps left by lost packets, and I prepared the hardware parts list. The sensor nodes generate a cyclone scenario, drop packets to emulate a lossy marine link, and the server repairs the missing sequence numbers. This report covers the design, the working code, my choices, updated costs, the problems I found and my contribution.

1. Detailed Design

Sensor node: every second the node increments its sequence number, updates wind, temperature and pressure (a small random walk in calm weather; after `--cyclone-after` seconds wind rises and pressure falls), builds a compact JSON record and, unless the loss draw says the packet is lost, sends it to the gateway. **Gap filling:** when the server sees a jump in sequence numbers, my `reconstruct_gap()` linearly interpolates each variable between the readings before and after the gap and marks the result as reconstructed (Figure 1).

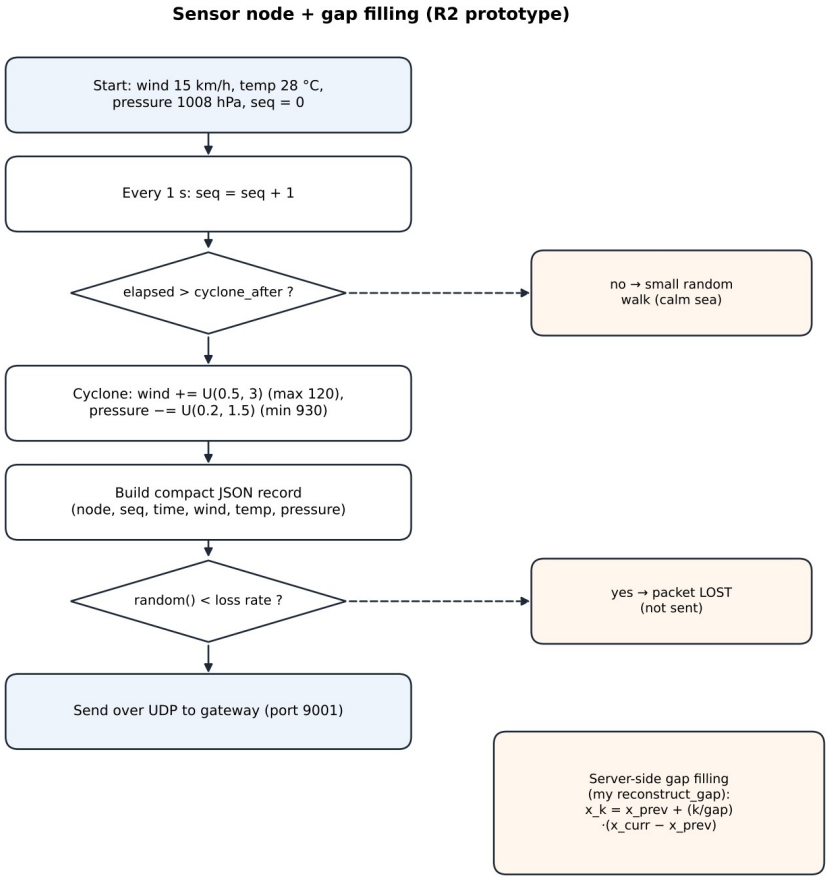


Figure 1: Flowchart of my sensor node and gap filling (prototype)

Table 1: Labelled parameters of my module

Parameter	Value	Meaning
--interval	1 s	Time between readings
--loss	0.08 default (0.10–0.15 in tests)	Probability that a packet is dropped
--cyclone-after	20 s default	When the storm starts in the scenario
Calm walk	wind ±1, pressure ±0.5, temp ±0.2 per step	Natural variation
Cyclone ramp	wind +0.5…3 (max 120), pressure −0.2…1.5 (min 930)	Approaching storm
Interpolation	x_k = x_prev + (k/gap)·(x_curr − x_prev)	Estimate for missing reading k

2. Actual Working Progress since Review 1

- **Code committed:** `sensor_node.py` (68 lines) and `reconstruct_gap()` in `central_server.py`, commit `82bcb8a`.

- **Running:** three simulated nodes (BUOY_07, RADAR_COAST_A, BUOY_12) with different loss rates and cyclone start times feed the gateway; the server marks filled readings as reconstructed.
- **Hardware:** parts list prepared (ESP32 + BMP280 per node); purchase planned after the software interface is fixed.

```
if random.random() < loss_rate:
    print(f"[{node_id}] seq={seq:05d} LOST (simulated link loss)")
else:
    sock.sendto(encode(record), (HOST, SENSOR_TO_GATEWAY_PORT))

def reconstruct_gap(node_id, prev_record, curr_record):
    gap = curr_record["seq"] - prev_record["seq"]
    for i in range(1, gap):
        frac = i / gap      # linear interpolation for each variable
```

To produce dated evidence for this report, the Review 2 prototype (repository commit `82bcb8a`, 16/9/2026) was re-run unchanged with three sensor nodes for 60 seconds; the logs are in `docs/reports/evidence/r2_prototype/`. The team's original 12-second test reported 66 packets received, 11 reconstructed and 2 critical, with overall risk 100 and the alert active.

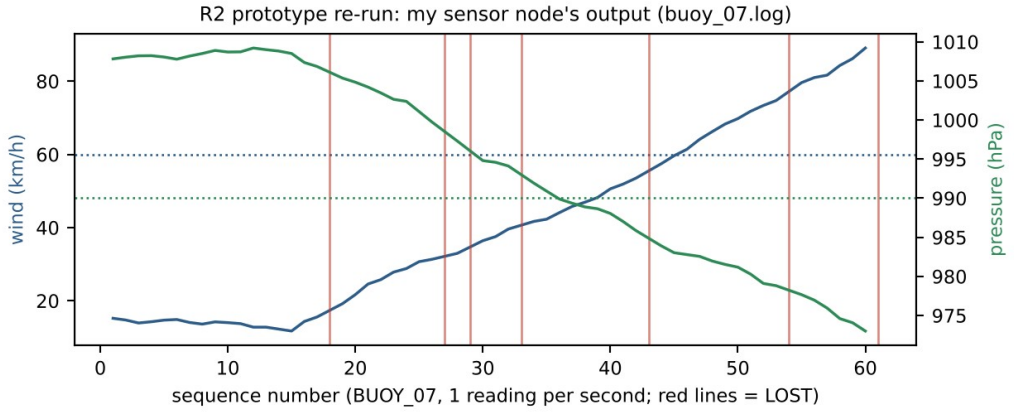


Figure 2: Output of my sensor node in the re-run; red lines mark lost packets

In the re-run my three nodes lost 22 packets in total; the server filled them using my function (22 reconstructed in `/status` at 60 s).

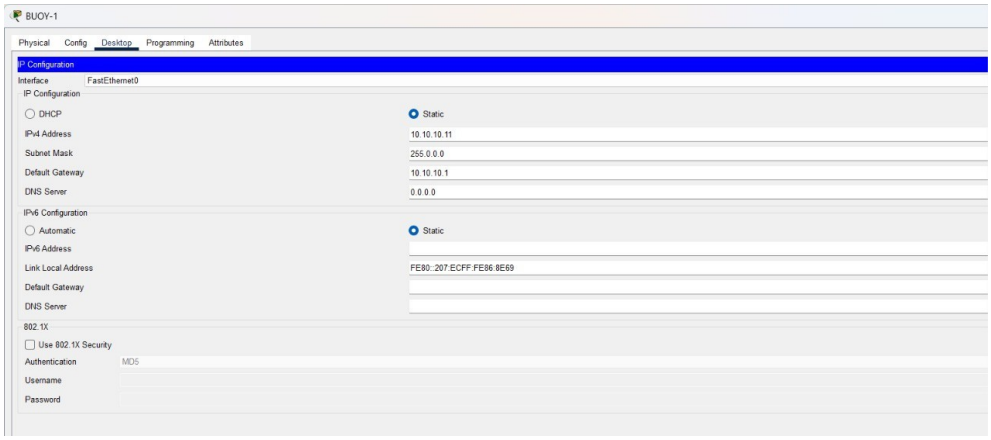


Figure 3: BUOY-1 end device in the team's Packet Tracer topology (joint work); it represents my sensor nodes in the routed network

Completion: about 40 % of my planned scope. **Done:** sensor simulator, cyclone scenario, loss emulation, interpolation baseline, parts list. **Remaining:** realistic storm physics and bursty loss, ground-truth logging, a learned reconstruction model, a satellite-station simulator, and the ESP32 nodes.

Table 2: Test runs performed on my module

Test run	Setup	Observation
Team test, 12 s, 2 nodes	BUOY_07 (15 % loss), RADAR_A (10 %)	11 lost packets filled by interpolation
Re-run, 60 s, 3 nodes	Prototype commit 82bcb8a	22 packets lost, all 22 reconstructed by the server
Burst analysis	Runs of LOST lines in the logs	Longest run 2 packets — losses too scattered
Storm-shape check	300 simulated steps of the node logic	Wind fixed at 120, pressure at 930 after ≈100 s
Unit check of reconstruct_gap()	Gap 10 → 13	Values correct, timestamps wrong (all 12:00:03)

3. Justification of Design Choices

Table 3: Comparison of alternatives for my module

Choice	Option	Approx. cost	Realism / accuracy	Power / ease	Decision
Data source	**Simulated nodes**	₹0	Controllable, known truth	Easiest	Chosen now
	Recorded buoy archives	₹0	Real but no control over loss	Needs data cleaning	Later
	ESP32 hardware nodes	≈₹800 / node	Real sensors, real Wi-Fi loss	Low power; build effort	Next phase
Gap filling	Hold last value	—	Poor when values change fast	Trivial	Rejected
	Linear interpolation	—	Good baseline; needs next reading	Easy	Chosen now
	LSTM / GRU-D	—	Can learn dynamics	Needs training	Next phase
Microcontroller	Arduino Uno + ESP8266	≈₹700	Works	Two boards	Rejected
	ESP32	≈₹400	Wi-Fi built in	Deep sleep available	Planned

4. Updated Feasibility

Table 4: Economic picture for my part

Aspect	Review 1 estimate	Now (Review 2)	Reason for change
Two hardware nodes	₹2,000–2,500	₹1,600–2,200	BMP280 (≈₹120) chosen instead of BME280; a potentiometer replaces a commercial anemometer for demos
Software	₹0	₹0	Python and Arduino IDE are free
Purchase status	—	Not yet bought	Waiting until the message format is final

Social: low-cost nodes would let local bodies and colleges densify coastal monitoring. **Environmental:** ESP32 boards are reusable and can sleep between readings; small packets mean less radio energy.

5. Problems Encountered and How I Adapted

Table 5: Problems in my module

Problem	How I diagnosed it	What I changed	Result
Simulated losses are unrealistically scattered	Counted runs of LOST lines in the re-run logs: 22 losses, the longest run only 2 packets; real storm links lose packets in bursts	Plan a two-state burst-loss (Gilbert–Elliott) model	Problem quantified; model chosen
The simulated cyclone	Simulated 300 steps of my node's	Plan a storm that approaches,	Needed to test alert

Problem	How I diagnosed it	What I changed	Result
never passes	logic: wind stays at 120 km/h and pressure at 930 hPa after about 100 s, so the alert can never clear	passes and recedes (physical wind/pressure profile)	clearing
Reconstructed readings had the wrong time	Unit check of reconstruct_gap(): readings 11 and 12 both got 12:00:03, the time of packet 13	Mark them as estimates of their own sequence slot; correct time stamping planned	Bug found before it affected results
Cannot measure how good the gap filling is	Lost readings are never recorded anywhere, so there is no truth to compare with	Plan a ground-truth log written by each node, including lost readings	Enables error measurement next phase

Plan from Review 2 to Review 3

Week	Planned work
15–21 Sep	Physics-based storm (approach, pass, recede) and burst-loss channel
22–28 Sep	Ground-truth logging; satellite-station simulator
29 Sep–5 Oct	Learned reconstruction model and training pipeline
6–9 Oct	ESP32 firmware and hardware guide; LoRa option

6. Individual Contribution Evidence

Table 6: Division of work

Tasks I did	Tasks done by teammates
Sensor-node simulator: readings, sequence numbers, cyclone ramp, loss emulation	Laksh: central server, REST API, dashboard
reconstruct_gap() interpolation used by the server	Anvay: edge gateway classification and priority forwarding
Hardware parts list and cost estimate; loss and gap analysis of the logs	Team: shared record format, Packet Tracer topology

- Code: `sensor_node.py` and `reconstruct_gap()` in `central_server.py`, commit `82bcb8a` (https://github.com/laksh-ahuja06/AI-DRIVEN-EDGE-CLOUD-NETWORK).
- Logs: `buoy_07.log`, `buoy_12.log`, `radar.log` and `server.log` in `docs/reports/evidence/r2_prototype/`.

> *[Insert here: screenshot of my own commits / work log and a photo or screen recording of me testing. Note: the team's early commits were pushed from one shared GitHub account, so the file-level split above is the record of who wrote what.]*

R2 Video Plan

Segment	What I show
Design walkthrough (2 min)	Face + screen; flowchart and parameter table
Live progress with my own work on screen (2 min)	Face + screen; code running and the logs
One problem and its solution (1 min)	Face + screen; the problem table and the evidence log
Updated cost and my contribution (1 min)	Face + screen; cost table and the division-of-work table